



I.4. QUALIDADE DE SOFTWARE

DISCIPLINA: ENGENHARIA DE SOFTWARE I
CURSO: BACHAREL EM SISTEMAS DE INFORMAÇÃO
PROFESSOR: M.E JULLIAN HERMANN CREUTZBERG
2026

Para você, o que é
qualidade?

Vamos montar uma lista....



QUALIDADE – O QUE É ?

- O que um determinado produto/serviço apresenta para considerarmos que tem qualidade?
- Ex.: Empresa Aérea (dados fictícios)

LATAM		GOL	
Pontualidade	8,0	Pontualidade	7,7
Serviço de bordo	7,0	Serviço de bordo	4,3
Check-in	8,1	Check-in	7,5
Conforto do avião	6,6	Conforto do avião	5,3
Entretenimento	5,8	Entretenimento	3,0
Cuidado com bagagem	7,1	Cuidado com bagagem	6,5
Atendimento dentro do avião	7,7	Atendimento dentro do avião	6,5
Custo-benefício	6,8	Custo-benefício	6,6
Média	6,9	Média	5,9

3

QUALIDADE – O QUE É ?

- Qualidade é um conceito relativo.
- Diversos aspectos são levados em conta.
- No caso de um automóvel, fatores como conforto, segurança, desempenho, beleza e custo têm estreita relação com a qualidade.

4

QUALIDADE – O QUE É ?

- Qualidade diz respeito à satisfação do cliente.
- “É atender plenamente os requisitos do cliente”
- “É superar a expectativa do cliente”

“A totalidade das características de uma entidade que lhe confere a capacidade de satisfazer às necessidades explícitas e implícitas”

(NBR ISO 8402)

5

QUALIDADE – VISÃO POPULAR

- Qualidade é vista como luxo, classe e questão de gosto;
- Produtos caros, sofisticados e mais complexos são considerados de maior qualidade que produtos similares mais simples;
- Qualidade pode ser discutida, sentida e julgada, mas não pode ser medida, controlada, nem gerenciada.

6

QUALIDADE – VISÃO PROFISSIONAL

- Qualidade está relacionada com os Requisitos designados para o produto;
- As não conformidades aos requisitos são consideradas defeitos, falta de qualidade;
- **Qualidade pode e deve ser medida, definida, monitorada, gerenciada e melhorada.**

7

VÍDEO COMPLEMENTAR

- Sugestão Vídeo: **Evolução Histórica da Gestão da Qualidade** (6 partes)
- <https://youtu.be/7pNpI52ZIIA>
- **Resumo:** O vídeo apresenta a evolução histórica da gestão da qualidade ao longo do tempo. Inicialmente, nas civilizações antigas, já existiam regras para garantir a qualidade de produtos e construções, como no Código de Hamurabi. Durante o período artesanal, o próprio produtor era responsável por todas as etapas do processo e pelo controle da qualidade. Com a Revolução Industrial e a produção em massa, surgiram métodos de inspeção para identificar defeitos nos produtos. No século XX, foram desenvolvidas técnicas de controle estatístico e surgiram especialistas na área. Assim, a qualidade deixou de ser apenas inspeção e passou a envolver toda a organização, com foco na melhoria contínua e na satisfação do cliente.

8

QUALIDADE

- Um fator negativo sobre qualidade é a **complexidade**: tamanho das especificações;
 - Construir um prédio de 10 andares é mais complicado que uma casa simples.
 - Mas a solução não é apenas aumentar o número de tijolos;
 - Em software isso é ainda mais grave, levando-se em conta as interações entre os diversos componentes do sistema;

9

QUALIDADE DE SOFTWARE

- Mudança tecnológica: o hardware evoluiu rapidamente, já o software é mais complexo;
- Os primeiros programadores não tinham ferramentas/métodos como hoje;
- Não havia sequer formação para programador.

10

SITUAÇÃO NAS EMPRESAS DE SOFTWARE

- Acúmulo de trabalho
- Abandono de planos e procedimentos
- Produto funciona, mas com defeitos; prazo e custo maiores; e menos funcionalidade
- Clientes e funcionários insatisfeitos
- Sucesso depende muito do esforço heroico das pessoas
- Pouca repetitividade

11

QUALIDADE DE SOFTWARE



- O que normalmente o cliente quer?
 - Atendimento aos requisitos especificados
 - Defeito zero
 - Alto desempenho
 - Baixo custo
 - Desenvolvimento rápido
 - Facilidade de uso
 - Eficiência nos serviços associados
 - Inovação

12

QUALIDADE DE SOFTWARE

- A qualidade de software depende principalmente do emprego correto dos métodos definidos para o processo.
- Embora sejam apoiados por várias ferramentas – ainda há uma grande lacuna manual.



13

- Caso você fosse realizar uma avaliação da qualidade da instituição de ensino UNIDAVI, quais itens fariam parte do levantamento?



14

- E quanto a qualidade de um software, o que iríamos avaliar?



15

QUALIDADE DE SOFTWARE

- Segundo PRESSMAN, é necessário:
 - Definir explicitamente o que é qualidade;
 - Criar atividades que garantam qualidade ao produto de software;
 - Realizar atividades de controle e garantia de qualidade;
 - Usar métricas para desenvolver estratégias para aperfeiçoar o processo de software;

16



GERENCIAMENTO DE QUALIDADE DE SOFTWARE

17

GERENCIAMENTO DE QUALIDADE DE SOFTWARE

- Preocupa-se em garantir que o nível necessário de qualidade seja alcançado em um produto de software.

18

GERENCIAMENTO DE QUALIDADE DE SOFTWARE

- Três principais preocupações:
 - No nível organizacional, o gerenciamento de qualidade se preocupa em **estabelecer um quadro de processos organizacionais e padrões que irão gerar um software de alta qualidade.**
 - No nível de projeto, o gerenciamento de qualidade envolve a **aplicação de processos de qualidade específicos e a verificação de que esses processos planejados sejam seguidos.**
 - No nível de projeto, o gerenciamento de qualidade também está preocupada com o **estabelecimento de um plano de qualidade para um projeto.** O plano de qualidade deve estabelecer as metas de qualidade para o projeto e definir os processos e padrões a serem usados.

19

ATIVIDADES DE GERENCIAMENTO DE QUALIDADE

- O gerenciamento de qualidade fornece uma verificação independente a respeito do processo de desenvolvimento de software.
- **O processo de gerenciamento da qualidade verifica as entregas do projeto para garantir que sejam consistentes com os objetivos e padrões organizacionais.**
- A equipe de qualidade deve ser independente da equipe de desenvolvimento para que possa ter uma visão objetiva do software.
- O que permite que façam relatórios sobre a qualidade do software, que não sejam influenciados por questões de desenvolvimento de software.

20

PLANEJAMENTO DE QUALIDADE

- Um plano de qualidade define as qualidades desejadas do produto e como esses são avaliados, além de definir os atributos de qualidade mais significativos.
- O plano de qualidade deve definir o processo de avaliação da qualidade.
- Ele deve estabelecer quais padrões da organização devem ser aplicadas e, se necessário, definir os novos padrões a serem usados.

21

PLANOS DE QUALIDADE

- Estrutura do plano de qualidade:
 - Introdução ao produto;
 - Planos de produto;
 - Descrições de processo;
 - Metas de qualidade;
 - Riscos e gerenciamento de riscos.
- Os planos de qualidade devem ser documentos curtos, sucintos.
- Se são muito longos, ninguém vai lê-los.

22

ÂMBITO DO GERENCIAMENTO DE QUALIDADE

- O gerenciamento de qualidade é particularmente importante para sistemas grandes e complexos.
- A documentação de qualidade é um registro do progresso e apoia a continuidade do desenvolvimento na medida em que a equipe de desenvolvimento muda.
- Para sistemas menores, o gerenciamento de qualidade necessita de menos documentação e deve se concentrar em estabelecer uma cultura de qualidade.

23

QUALIDADE DE SOFTWARE

- **De uma maneira simplista, a qualidade significa que um produto deve corresponder às suas especificações.**
- O que é problemático para os sistemas de software.
 - Existe uma tensão entre os requisitos de qualidade do cliente (eficiência, confiabilidade, etc.) e os requisitos de qualidade do desenvolvedor (reuso, de manutenção, etc.);
 - Alguns requisitos de qualidade são difíceis de se especificar de forma inequívoca;
 - Geralmente as especificações de software são incompletas e muitas vezes inconsistentes.
- O foco pode ser "adequação à finalidade" em vez de conformidade à especificação.

24

ADEQUAÇÃO DO SOFTWARE À FINALIDADE

■ Algumas perguntas básicas:

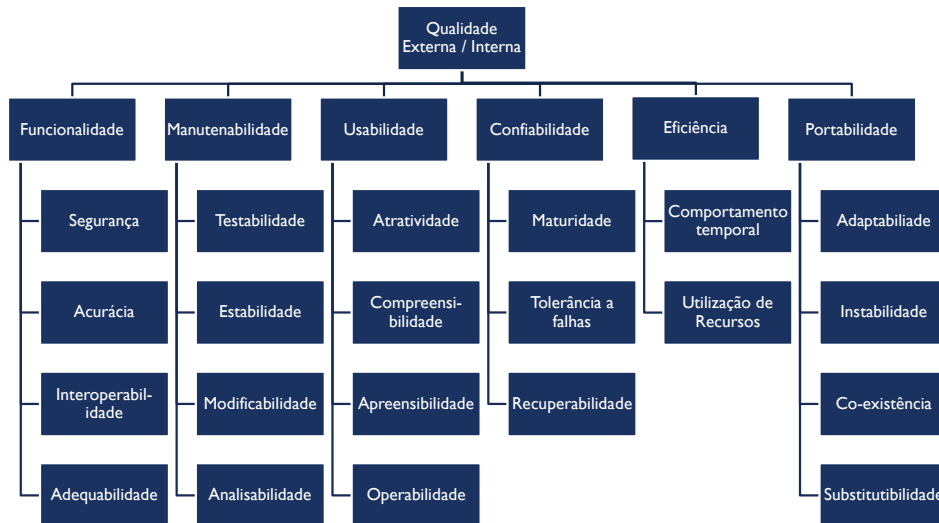
- Durante o processo de desenvolvimento os padrões de programação e documentação foram seguidos?
- O software foi devidamente testado?
- O software é confiável o suficiente para ser colocado em uso?
- O desempenho do software é aceitável para uso normal?
- O software é usável?
- O software bem é compreensível e estruturado?

25

ATRIBUTOS DE QUALIDADE DE SOFTWARE

Segurança	Compreensibilidade	Portabilidade
Proteção	Testabilidade	Usabilidade
Confiabilidade	Adaptabilidade	Reusabilidade
Resiliência	Modularidade	Eficiência
Robustez	Complexidade	Capacidade de aprendizado

26



27

CONFLITOS DE QUALIDADE

- Não é possível para qualquer sistema ser otimizado para todos esses atributos – por exemplo, melhorar a robustez poderá levar à perda de desempenho.
- **Portanto, o plano de qualidade deve definir os atributos de qualidade mais importantes para o software que está sendo desenvolvido.**
- O plano também deve incluir uma definição do processo de avaliação de qualidade, uma forma acordada de avaliar se alguma qualidade, como a de manutenção ou robustez, está presente no produto.

28

QUALIDADE DE PROCESSO E PRODUTO

- **A qualidade de um produto desenvolvido é influenciada pela qualidade do processo de produção.**

- Isso é importante no desenvolvimento de software pois alguns atributos de qualidade de produto são difíceis de avaliar.
- No entanto, existe uma relação muito complexa e mal compreendida entre os processos de software e a qualidade de produto.
- A aplicação das habilidades individuais e experiência é particularmente importante no desenvolvimento de software;
- Fatores externos, tais como a novidade de uma aplicação ou a necessidade de um cronograma de desenvolvimento acelerado pode prejudicar a qualidade de produto.

29

QUALIDADE DE SOFTWARE

- Dois aspectos principais podem ser considerados na qualidade de um sistema computacional:
 - O próprio produto de software gerado;
 - O processo através do qual ele é produzido.



30

QUALIDADE DE SOFTWARE - PRODUTO

- “Conformidade a requisitos funcionais e de desempenho explicitamente declarados, a padrões de desenvolvimento claramente documentados e a características implícitas que são esperadas de todo software **profissionalmente** desenvolvido”. (PRESSMAN)

31

QUALIDADE DE SOFTWARE

- Consiste na qualidade do:
 - **Produto** → funcionalidade, usabilidade, confiabilidade, eficiência, manutenibilidade e portabilidade
 - **Processo** → dos requisitos até a entrega do produto final há um processo de desenvolvimento.

**O processo influencia na
qualidade do Produto!**

32



PADRÕES DE SOFTWARE

33

PADRÕES DE SOFTWARE

- Os padrões definem os atributos necessários de um produto ou processo.
- Eles desempenham um papel importante no gerenciamento de qualidade.
- Os padrões podem ser internacionais, nacionais, padrões organizacionais ou de projeto.
- Os padrões de produto definem características que todos os componentes de software devem exibir, por exemplo, um estilo de programação comum.
- Os padrões de processo definem como o processo de software deve ser seguido.

34

IMPORTÂNCIA DOS PADRÕES

- Uma síntese das melhores práticas evita a repetição de erros do passado.
- São um framework para definir o que significa a qualidade de uma determinada configuração, ou seja, a visão de qualidade daquela organização.
- Eles fornecem a continuidade – pessoas novas podem compreender a organização através da compreensão dos padrões usados.

35

PADRÕES DE PRODUTO E DE PROCESSO

Padrões de produto	Padrões de processo
Formulário de revisão de projeto	Condução de revisão de projeto
Estrutura de documento de requisitos	Apresentação do novo código para a construção de sistema
Formato de cabeçalho de método	Processo de versão e release
Estilo de programação Java	Processo de aprovação de plano de projeto
Formato de plano de projeto	Processo de controle de mudança
Formulário de solicitação de mudança	Processo de registro de teste

36

PROBLEMAS COM OS PADRÕES

- Podem não ser percebidos como relevantes e atualizados pelos engenheiros de software.
- Muitas vezes eles envolvem o preenchimento de formulários muito burocráticos.
- Se não recebem apoio de ferramentas de software, geralmente, o preenchimento tedioso de formulários é acrescentado para manter a documentação associada com os padrões.

37

DESENVOLVIMENTO DE PADRÕES

- Envolver os profissionais no desenvolvimento.
- Os engenheiros devem compreender as razões subjacentes a um padrão.
- Revisões regulares dos padrões e seu uso.
- Os padrões podem se tornar desatualizados rapidamente o que reduz a sua credibilidade entre os profissionais.
- Padrões detalhados devem ter suporte de ferramentas especializadas.
- O trabalho burocrático excessivo é a queixa mais significativa aos padrões.
- Formulários baseados na web não são bons o suficiente.

38



REVISÕES E INSPEÇÕES

39

REVISÕES E INSPEÇÕES

- **Um grupo examina parte ou a totalidade de um processo ou sistema e sua documentação para encontrar potenciais problemas.**
- Softwares ou documentos podem ser "assinados" em uma revisão, o que significa que o avanço para a fase seguinte de desenvolvimento foi aprovado pela gerência.

40

AVALIAÇÕES DE QUALIDADE

- Existem diferentes tipos de revisão com objetivos diferentes
 - Inspeções para remoção de defeitos (produto);
 - Comentários para avaliação de progresso (produto e processo);
 - Avaliações de qualidade (produto e padrões).
- Código, projetos, especificações, planos de teste, padrões, etc., todos podem ser revistos.

41

COMENTÁRIOS E MÉTODOS ÁGEIS

- Geralmente, o processo de revisão no desenvolvimento ágil de software é informal.
 - Em Scrum, por exemplo, há uma reunião de avaliação após cada iteração do software ser concluída (uma revisão sprint), na qual as questões de qualidade e problemas podem ser discutidos.
 - No Extreme Programming (XP), a programação em pares garante que o código seja constantemente examinado e revisado por outro membro da equipe.
 - XP se baseia em pessoas que tomam a iniciativa de melhorar e refatorar o código.
 - As abordagens ágeis geralmente não são orientadas por padrões, para que os problemas de compatibilidade com os padrões normalmente não sejam considerados.

42

INSPEÇÕES DE PROGRAMA

- **Essas são avaliações em pares, em que os engenheiros examinam a fonte de um sistema com o objetivo de descobrir anomalias e defeitos.**
- As inspeções não exigem a execução de um sistema, assim podem ser usadas antes da implementação.
- Elas podem ser aplicadas a qualquer representação do sistema (requisitos, projeto, configuração, dados de teste, etc.)
- Elas têm se mostrado uma técnica eficaz para descobrir bugs de programas.

43

CHECKLISTS DE INSPEÇÃO

- Um checklist de inspeção dos defeitos comuns deve ser usada para conduzir a inspeção.
- Os checklists de inspeção de defeitos são dependentes da linguagem de programação e refletem os erros característicos que podem surgir na linguagem.
- Em geral, quanto "mais fraca" a verificação de tipos, maior o checklist.
- Exemplos: Iniciação, nomeação de constantes, repetição de loop, limites de vetor, etc.

44

UM CHECKLIST DE INSPEÇÃO (A)

Classe de defeito	Verificação de inspeção
Defeitos de dados	• Todas as variáveis de programa são iniciadas antes que seus valores sejam usados? • Todas as constantes foram nomeadas? • O limite superior de vetores deve ser igual ao tamanho do vetor ou ao tamanho -1? • Se as <i>strings</i> de caracteres são usadas, um delimitador é explicitamente atribuído? • Existe alguma possibilidade de <i>overflow</i> de <i>buffer</i>?
Defeitos de controle	• Para cada instrução condicional, a condição está correta? • É certo que cada <i>loop</i> vai terminar? • As declarações compostas estão posicionadas corretamente entre colchetes? • Em declarações <i>case</i>, todos os <i>cases</i> possíveis são considerados? • Se um <i>break</i> é requerido após cada <i>case</i> em declarações <i>case</i>, este foi incluído?
Defeitos de entrada/saída	• Todas as variáveis de entrada são usadas? • Todas as variáveis de saída receberam um valor antes de serem emitidas? • Entradas inesperadas podem causar corrupção de dados?

45

UM CHECKLIST DE INSPEÇÃO (B)

Defeitos de interface	• Todas as chamadas de funções e métodos têm o número correto de parâmetros? • Os parâmetros formais e reais correspondem? • Os parâmetros estão na ordem correta? • Se os componentes acessam memória compartilhada, eles têm o mesmo modelo de estrutura de memória compartilhada?
Defeitos de gerenciamento de armazenamento	• Se uma estrutura ligada é modificada, todas as ligações foram corretamente reatribuídas? • Se o armazenamento dinâmico é usado, o espaço foi alocado corretamente? • O espaço é explicitamente desalocado após não ser mais necessário?
Defeitos de gerenciamento de exceção	• Foram levadas em consideração todas as condições possíveis de erro?

46

MÉTODOS ÁGEIS E INSPEÇÕES

- Processos ágeis **raramente** usam inspeção formal ou processos de revisão em pares.
- Em vez disso, eles contam com membros da equipe cooperando para controlar os código uns dos outros, e as orientações informais, tais como 'verifique antes do check-in', que sugerem que os programadores devem verificar o seu próprio código.
- Os adeptos do Extreme Programming argumentam que a programação em pares é um substituto eficaz para a inspeção, pois é, com efeito, um processo de inspeção contínua.
- Duas pessoas olham para cada linha de código e verificam essa antes que seja aceita.

47

CUSTO DA QUALIDADE

48

QUALIDADE DE SOFTWARE

- Gestão de qualidade abrange:
 - Processo de garantia de qualidade;
 - Tarefas específicas de garantia de qualidade;
 - Prática de engenharia de software (métodos e ferramentas);
 - Controle dos produtos de software e suas modificações;
 - Procedimento para garantir satisfação de normas (quando existir);
 - Mecanismos de medição e relatório;

49

QUALIDADE DE SOFTWARE

Custo da Qualidade:

- Qualidade envolve custos que engloba todas as atividades para busca da qualidade
- São divididos em:
 - Custos com a prevenção;
 - Com a avaliação;
 - E com as falhas;



50



51

QUALIDADE DE SOFTWARE

- Importante: o custo do produto aumenta exponencialmente, quanto mais tarde no ciclo de vida o defeito for descoberto



52

QUALIDADE DE SOFTWARE

- **Garantia de Qualidade:**
- Conjunto de funções para auditar e relatar a efetividade e completeza das atividades do controle de qualidade;
- Fornecer dados à gerencia sobre a qualidade do produto;

53

QUALIDADE DE SOFTWARE

- Garantia de Qualidade de Software:
- Podemos redefinir a qualidade de software como: Conformidade com requisitos funcionais e de desempenho explicitamente declarados;
 - Se um requisito não for satisfeito, não há qualidade;
- Não é feito somente no final do projeto;

54



DEFEITO, ERRO OU FALHA

55

DEFEITO, ERRO OU FALHA



56

DEFEITO, ERRO OU FALHA

- **Defeito:** Uma imperfeição do produto;

- Ex:

```
a : integer;  
b : integer;  
c : integer;  
readln a;  
readln b;  
c = a / b;
```

Esse código pode representar um defeito, pois a divisão poderá resultar em um erro de divisão por zero.

57

DEFEITO, ERRO OU FALHA

- **Erro:** Um erro pode ser gerado a partir de um defeito existente na codificação.
- Se o usuário passar por esse **erro**, o software irá se comportar de maneira incorreta gerando assim uma falha.

58

DEFEITO, ERRO OU FALHA

- **Falha:** Falha é o resultado errado provocado por um defeito ou condição inesperada;
- O código anterior é um defeito, mas o programa pode funcionar durante anos sem que apresente o defeito;
- Falhas também podem ocorrer por fatores externos ao programa, como uma base de dados corrompida.

59

ATIVIDADE

60

ATIVIDADE

- Atividade individual ou em duplas.
- Cada um deverá construir um guia de qualidade de software contendo no mínimo 5 subcaracterísticas a serem avaliadas específicas sobre o sistema avaliado. Selecione as mais relevantes considerando o contexto do software proposto.
- As subcaracterísticas devem ter como base as características de qualidade citadas – ler com atenção todas as características primeiro.
- Deve ser construído conforme o exemplo, explicando detalhadamente como você faria para avaliar/mensurar a cada um das subcaracterísticas selecionadas.
- Contexto: **Aplicativo de Registro de Ponto**
- 50 min.
- 1,0 ponto na Nota de Atividades

61



62

Característica: Funcionalidade (F)		
Capacidade para o cumprimento de tarefas.		
<i>Devem ser avaliados os requisitos funcionais do sistema.</i>		
Sub-Característica: Segurança de Acesso (S)		
F01-S	O acesso às funções do programa deverá ser permitida somente a usuários autorizados. Deve ser testada a concessão/retirada de privilégios para o usuário.	Essencial
Sub-Característica: Acurácia (C)		
F01-C	Devem ser consistidos os resultados apresentados pelo programa em suas mais diversas saídas: Consulta, Tela de Manutenção (em cálculos de agrupamento), Relatórios, Arquivos de Exportação / Importação.	Depende do tipo de programa
Sub-Característica: Interoperabilidade (I)		
F01-I	As Integrações do programa com outros programas / módulos e/ou rotinas devem ser consistidas. Neste ponto todos os processos que devem ser disparados por este, são de teste obrigatório.	Depende do tipo de programa
Sub-Característica: Adequabilidade (Q)		
F01-Q	Caso existam parâmetros que contemplem o funcionamento do programa, estes devem ser testados. Deverá ser verificado para isto o escopo dos parâmetros, situação em que os mesmos podem influenciar outros programas e/ou módulos.	Depende do tipo de programa

63

REFERÊNCIAS

- PRESSMAN, Roger S. Engenharia de Software. São Paulo: Makron Books, 1995.
- SOMMERVILLE, Ian. Engenharia de Software. 9. ed. São Paulo: Pearson, 2011.

64



jullian@unidavi.edu.br



@jullianhc

65